Final Report on Applied Machine Learning Project - Machine Translation

Martina Kamodyova, Sandra Poernbacher, Zhiyi Chen

Introduction

Machine translation is a critical area within multilingual technologies and closely aligns with our study's focus. As individuals with a language background, we find it particularly compelling to examine translation from a technical perspective and to explore various translation models. Given the increasing reliance on machine translation in today's globalized world, it is essential to understand both its strengths and limitations.

For this project, we chose TED Talks as our primary data source due to the availability of parallel corpora spanning multiple language pairs, all human-translated and revised. TED Talks uniquely blend features of written and spoken communication, offering a diverse range of topics delivered by experts to a broad and curious audience.

The overarching vision of TED is to spread outstanding ideas worldwide, with multilingualism playing a vital role in making high-quality educational content accessible to a global audience, especially non-English speakers. By supporting inclusiveness and providing translated content for linguistically diverse and underserved communities, TED ensures that valuable knowledge reaches as many people as possible.

Currently, TED relies on volunteers to translate and revise talks, a process that is both time-consuming and labor-intensive. High-quality machine translation has the potential to accelerate this workflow significantly. Furthermore, a translation model trained extensively on TED Talks could be adapted for use in related fields, such as popular science communication, thereby extending its impact.

For this project, we selected the WIT TED Talks dataset. After a series of data preparation steps, we created a bicleaned version to compare translation quality before and after bicleaning. We began by testing several ready-to-use models via the translation pipeline to obtain initial results. Based on these observations, we fine-tuned the T5-base and NLLB-200 models on our training data. Finally, we developed a model-agnostic training pipeline with added hyperparameters to support future fine-tuning and experimentation. Our preprocessed datasets and code source are available at https://github.com/kammartina/ML_Applied_Project_2025S.

Data Description

Considering the diverse language backgrounds in our team (German, English, French, Italian, Chinese, and Slovak), we prioritized locating a multilingual dataset that could support most of these languages. After extensive search, we selected the **"IWSLT/ted_talks_iwslt"** dataset from the Hugging Face library. This dataset includes TED talk transcripts and volunteer-based translations in over 100 languages.

In this dataset, each language is stored in a separate XML file containing subtitled versions of the talks. Among the dataset features, there is "<content>" section containing the full text of the talk. Another section is the "<transcript>" section containing sequentially ordered sentence fragments, which are wrapped in "<seekvideo>" tags with unique IDs. We chose to work with the “<transcript>” segments because they are well-aligned across languages, which is critical for building a multilingual model.

Preprocessing

To prepare the multilingual dataset for our multilingual translation task, we implemented a multi-stage data preprocessing pipeline. This process ensured the data was clean, aligned, and in a format suitable for training transformer-based models. We initially focused on the English-German language pair and later extended the process to include French. The source code for the dataset preprocessing called “dataset_preprocessing.py” can be found on GitHub. Below is a detailed breakdown of each step.

The first step involved parsing the original XML files, where we extracted the segmented frames from “<transcript>” section along with the corresponding “talk_id”. These were saved in language-specific CSV files. To improve accessibility and structure, we decided to convert the CSV files into JSON format, and work rather with this format.

A critical step was aligning the language-specific JSON files by intersecting them based on “talk_id”, ensuring that only talks transcripts available in both languages were retained. This resulted in a new structure, an aligned English-German JSON file, where each entry contains a pair of segment lists – one in English, one in German – ensuring consistent alignment for training.

The last structural transformation of the dataset complies with Hugging Face‘s recommended input format for translation tasks, and therefore we transformed the aligned dataset into JSON lines format. In this structure, each line becomes an object which contains a “translation” key pointing to a dictionary with “en” and “de” keys:

{"translation": {"en": "We have confusion about what we should think about a new technology.", "de": "Wir sind verwirrt über das, was wir über neue Technik denken sollten."}}

The aligned dataset in the JSON lines format was split into three subsets – training, validation and test set. These splits were saved as separate files to support model training of various pre-trained transformer models. This configuration was used as the reference setup for evaluating transformer-based models in our experiments.

To further enhance translation quality, we applied Bicleaner, a statistical tool designed to detect and remove noisy or misaligned sentence pairs in bilingual corpora. Bicleaner evaluates sentence pairs using a trained classifier and filters out low-confidence matches. Using this tool we obtained a cleaner, higher-quality parallel dataset, which we then used to fine-tune our selected models. This allowed us to compare translation performance between the reference setup and cleaned versions of the dataset.

We repeated the entire preprocessing pipeline for the English-French language pair, following the same steps: XML parsing, JSON conversion, segment alignment, transformation into JSON Lines format, and cleaning with Bicleaner. For future projects, this process could be further optimized by creating a unified multilingual dataset that contains aligned segments across all target languages. This would allow for more efficient fine-tuning of multilingual models and enable broader cross-lingual capabilities within a single training framework.

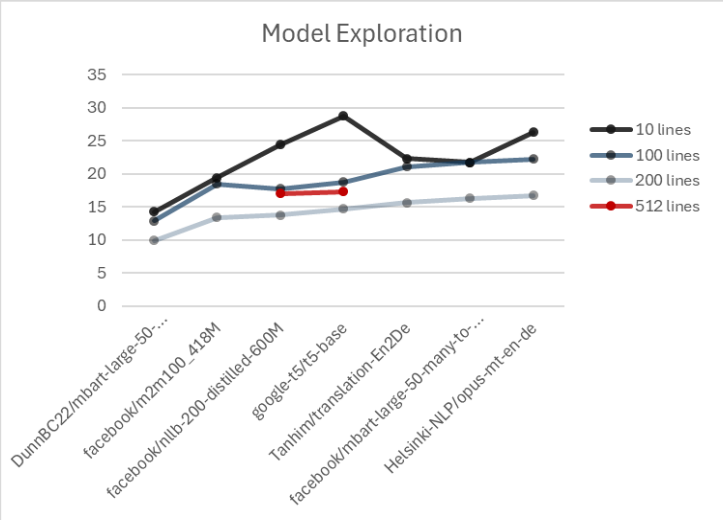
Model Exploration

To conduct a preliminary evaluation of translation performance from English to German, we selected the following pre-trained, transformer-based models from Hugging Face that support direct translation via the "translation" pipeline:

Model	Model Type	Model Size (no. of parameters)	Pipeline task, source code, target code
google-t5/t5-base	t5	222M	"translate_en_to_de", “en”, “de”
Helsinki-NLP/opus-mt-en-de	marian	74M	"translation", “en”, “de”
Tanhim/translation-En2De	marian	74M	"translation", “en”, “de”
facebook/nllb-200-distilled-600M	nllb	615M	"translation", “eng_Latn”, “deu_Latn”
facebook/mbart-large-50-many-to-many-mmt	mbart	610M	"translation", “en_XX”, “de_DE”
DunnBC22/mbart-large-50-English_German_Translation	mbart	610M	"translation", “en_XX”, “de_DE”
facebook/m2m100_418M	m2m100	483M	"translation", “en”, “de”

We evaluated a mix of base models (T5, NLLB, mBART, M2M) and fine-tuned models for English-to-German translation. Tests were conducted on varying numbers of segments from our pre-bicleaned test set to assess performance. We developed a script to accommodate different task specifications in *pipeline()*, along with the varying requirements for source and target language codes across models, see table above.

Translation quality was measured using BLEU scores, which generally decreased as the number of lines translated increased from 10 to 200. Given our objective of training a multilingual model, we selected two models of significantly different sizes, *T5-base* and *NLLB-200-distilled-600M* for fine-tuning on our training set. Evaluated on the full 512-line test set, both models showed improved BLEU scores compared to translating 200 lines, with *T5-base* reaching 17.3 (+17%) and *NLLB-200* 17.05 (+24%).



Fine-Tuning

Text-to-Text-Transfer Transformer (T5)

We conducted a series of fine-tuning experiments using the pre-trained T5-base model from Hugging Face library for English-to-German and English-to-French translation tasks. Our goal was to evaluate the impact of data quality and multilingual training on translation performance.

The T5-base model is a transformer-based model developed by Google in 2019. Its base version has approx. 220 million parameters, and the model was pre-trained on the C4 corpus using a span corruption objective. It has 12 encoder and 12 decoder layers. For translation, we used the task-specific prefix format which the model uses: "translate English to German: "; "translate English to French: ", and defined the source and target languages.

The code for fine-tuning process was implemented in two separate Python scripts, called “t5_base_en-de.py” and t5_base_en-fr.py”. We conducted a series of fine-tuning experiments using the pre-trained T5-base model from Hugging Face for translation tasks involving English-to-German and English-to-French language pairs. In total, three distinct versions of the T5-base model were fine-tuned under varying dataset conditions to evaluate the impact of data quality and multilingual scope on translation performance.

We used the *T5ForConditionalGeneration* class from the Hugging Face Transformers library, which is designed for sequence-to-sequence tasks such as translation. Fine-tuning was implemented using the *Seq2SeqTrainer* class, a high-level training loop specifically optimized for sequence generation tasks. Training configurations were defined using the *Seq2SeqTrainingArguments* class, which allowed us to control key hyperparameters. In order to reduce the computational power and training speed, we worked with a subset of data – 32000 training, 512 validation and 512 test samples of the particular dataset splits. The models were trained for 4 epochs using a batch size of 16 and a learning rate of 2e-5. All models were trained using the *SentencePiece* tokenizer and a maximum input length of 250 tokens, with beam search (5 beams) enabled during generation to improve translation diversity and quality. The whole training was performed in a Google Colab environment because of the problems that have arisen with the performance and storage of the university server.

Three main fine-tuned models were created:

- Model #1: „kammartina/t5_base_en-de_finetuned_32K" – This model was trained on the unclean English-German dataset.
- Model #2: „kammartina/t5_base_en-de_finetuned_32K_cleaned_dataset" – This model was trained on the clean English-German dataset cleaned with *Bicleaner*.
- Model #3: „kammartina/t5_base_en-de-fr_finetuned_32K_cleaned_dataset" – This model was trained on the unclean English-German and English-French dataset cleaned with *Bicleaner*.

During training, model checkpoints were saved at regular intervals, but only the best-performing checkpoint was retained for evaluation. This selected checkpoint was then used to generate predictions on the validation set and to perform the final evaluation on the test set. For all evaluations, we used the *SacreBLEU* metric to measure translation quality. The same procedure was applied to each of the fine-tuned models. The final versions of these models were uploaded to the Hugging Face Model Hub and are publicly available for reproducibility and further use. To see more detail concerning the fine-tuned T5 models, see the *Attachment 1*.

Text-to-Text-Transfer Transformer (T5) - Results

To evaluate translation performance, three variants of the T5-base model were fine-tuned and tested using the SacreBLEU metric. Here is an overview table (for more detail see *Attachment 1*):

	Model #1	Model #2	Model #3
	„kammartina/t5_base_en-de_finetuned_32K"	„kammartina/t5_base_en-de_finetuned_32K_cleaned_dataset"	„kammartina/t5_base_en-de-fr_finetuned_32K_cleaned_dataset"
	Epoch 3	Epoch 4	Epoch 4
Training loss	**1.79**	**1.71**	**1.29**
Training runtime (s)	1918.3	1926.7	1888.6

Samples/ sec	66.7	66.4	67.8
Validation SacreBLEU	**-**	**22.36**	**32.69**
Validation loss	-	1.669	1.195
Validation generatiog length	-	24.7	30.5
Test SacreBLEU	**22.09**	**24.24**	**27.92**
Test loss	1.710	1.580	1.325
Test generation length	25.6	25.9	34.1

These results show that cleaning parallel data with tools like *Bicleaner* enhances translation quality, and that T5-base model generalizes well across language pairs. In the direction of English to German, the BLEU score increased from 22.09 to 24.24 and the test loss lowered. The multilingual T5-base model #3 outperformed both models #1 and #2 in both BLEU and loss. However, we observed a significant difference of the model #3 between the BLEU score of validation and test, from 32.69 to 27.92. This noticeable drop can be attributed to a few common factors like random sampling variance, sentence length distribution or BLEU sensitivity to extract n-gram matches. Another reason could be a slight overfitting to patterns in the validation set.

In further chapters, we use the *compare-mt* tool for deeper error analysis and compare all models (including the models from the next chapter) in the translation directions English→German and English→French.

NLLB 200 distilled 600M

For our multilingual translation experiments, we also fine-tuned Meta's NLLB-200-distilled-600M model using the Hugging Face Transformers library. The model is a compact, distilled version of the full NLLB-200 and supports translation between 200 languages. We focused on English→German and English→French translation tasks using sentence-aligned TED Talk data.The model was chosen due to its multilingual capabilities and manageable size, which make it suitable for experimentation within limited GPU resources. The model requires explicit language codes for each translation direction. Therefore, we used *eng_Latn* for English, *deu_Latn* for German and *fra_Latn* for French. This was necessary during both tokenization and generation.

We used *AutoTokenizer* and *AutoModelForSeq2SeqLM* classes and wrapped the training loop in a *Seq2SeqTrainer*. Training was conducted using FP16 mixed precision to utilize GPU memory efficiently. We also implemented *DataCollatorForSeq2Seq* for dynamic padding and evaluation using SacreBLEU and chrF scores through the Hugging Face *evaluate* module. To handle the data preparation efficiently and maintain reproducibility, we modularized preprocessing and tokenization into a dedicated loader file, allowing streamlined integration into training scripts. Data was loaded from *.jsonl* files with the translation key structure *{"translation":{"en": …, "de": …}}* and we extracted language pairs accordingly.

Our training experiments were designed around three model variants:

- Model #1 (nllb_en-de_finetunded): Trained on unclean English→German data to serve as a baseline

- Model #2 (nllb_en-de_finetuned_cleaned_32K): Trained on a Bicleaner-filtered English→German dataset to examine the impact of noise reduction

- Model #3 (nllb_en-fr_finetuned_cleaned_32K): Trained on a cleaned English-French dataset to assess multilingual transfer learning

Each model was trained using 32000 examples for training and 512 for both validation and test. A maximum input length of 250 tokens and a batch size of 16 were selected to balance GPU capacity with sequence coverage. We trained for 5 epochs each. The use for a relatively large training size aimed to ensure sufficient learning signal for the low-resource NLLB variant while enabling manageable training time. Model checkpoints were saved at each evaluation epoch to monitor progress and prevent overfitting. The best checkpoint based on validation metrics was selected for final evaluation on the test set.

The following table summarizes our results (for more details see *Attachment 2*):

	Model #1	Model #2	Model #3
	sanpoer/nllb_200_distilled_en-de_finetuned	sanpoer/nllb_200_distilled_en_de_finetuned_cleaned32K	sanpoer/nllb_200_distilled_en-fr_finetuned_cleaned_32K
	epoch 4	epoch 5	epoch 5
Training loss	**0.96**	**0.99**	**0.41**
Training runtime (s)	5225.74	4978.36	5243.07
Samples/sec	30.62	32.14	30.52
Validation SacreBLEU	**21.37**	**22.80**	**1.03**
Validation chrF	**45.80**	**46.84**	**15.07**
Validation loss	0.71	0.72	0.91
Validation generatiog length	-	24.7	30.5
Test SacreBLEU	**23.25**	**25.20**	**32.34**
Test chrF	**46.58**	**49.43**	**53.44**
Test loss	0.79	0.73	0.58
Test generation length	25.6	25.9	34.1

Cleaning the English→German dataset with Bicleaner provided modest but consistent improvements in BLEU and chrF scores. These improvements validate the assumption that translation quality benefits from clean data. The English→French model (#3) was trained sequentially after the German model, showcasing promising generalization and transfer performance with a test BLEU of 32.34 despite low validation BLEU. This suggests the French set had a different structure or domain distribution compared to validation samples.

Fine-tuning NLLB-200-distilled-600M proved effective for domain-specific TED talk translation. The model responded well to dataset cleaning and showed promising transfer capabilities in multilingual settings. The modular data pipeline and clear logging helped in tracking model behavior across experiments. Overall, the NLLB model is a versatile multilingual system that benefits from clean data, stable batch sizing and careful language code management. With additional tuning or curriculum learning strategies, even better results could potentially be achieved across broader language pairs.

Models Comparison

English to German

To evaluate the translation performance of the fine-tuned translation-based models, each model was evaluated using the SacreBLEU metric on the test set, and further analyzed using the *compare-mt* tool for deeper insights. Tables with all results are provided in the *Attachment 3*. Some of the results are explained below. The source code for this part of the report called be found on GitHub. The following references to the particular models were used:

- sys1: T5 model #1: „kammartina/t5_base_en-de_finetuned_32K“
- sys2: T5 model #2: „kammartina/t5_base_en-de_finetuned_32K_cleaned_dataset“
- sys3: T5 model #3: „kammartina/t5_base_en-de-fr_finetuned_32K_cleaned_dataset“
- sys4: NLLB model #1: sanpoer/nllb_200_distilled_en-de_finetuned
- sys5: NLLB model #2: sanpoer/nllb_200_distilled_en_de_finetuned_cleaned32K
- sys6: NLLB model #3: sanpoer/nllb_200_distilled_en-fr_finetuned_cleaned_32K

1. Aggregate BLEU scores

The overall BLEU scores showed clear performance differentiation:

	sys1	sys2	sys3	sys4	sys5	sys6
SacreBLEU score	**19.17**	18.43	18.43	17.89	16.53	16.54

v s1 / s2 ->	sys2	sys3	sys4	sys5	sys6
sys1	-(p=0.23)	-(p=0.23)	-(p=0.23)	-(p=0.10)	-(p=0.15)
sys2		**-(p=1.00)**	-(p=0.38)	-(p=0.20)	-(p=0.25)
sys3			-(p=0.38)	-(p=0.20)	-(p=0.25)
sys4				**-(p=0.09)**	-(p=0.25)
sys5					-(p=0.48)

The p-value may be interpreted as a confidence score. The improvement in BLEU in sys1 vs. sys2/sys3 (p=0.23) is very small and not very significant. Comparison between sys1 and sys5/sys6 (p=0.10/0.15) points to a moderate confidence that sys1 performs better. Systems sys2 and sys3 (p=1.00) are functionally identical in performance. The closest to statistical significance in the entire set are sys4 and sys5 (p=0.09), suggesting that sys4 may truly be better than sys5. It needs to be pointed out that small BLEU differences (<1 point) should not be interpreted as meaningful unless statistically significant.

The highest BLEU score (19.17) was achieved by sys1, although differences with cleaned variants were not statistically significant (e.g., p=0.23 between sys1 and sys2). This suggests that while cleaning improved some qualitative aspects (as could be seen in word-level accuracy), its impact on overall BLEU may depend on domain specifics or sentence length. BLEU is also sensitive to exact match, and therefore, the statistical tests must be paired with qualitative analysis. BLEU alone doesn’t capture improvements in fluency, terminology, or rare word handling.

2. Sentence length and BLEU

This following table provides the BLEU scores given the sentence lengths. The table reveals that BLEU performance varies significantly with sentence length, and a single overall score does not capture these nuances. The highest BLEU scores are observed in the longer sentences (40-50 tokens). In this case, sys2 and sys3 dominate with 32.28 BLEU, indicating they handle more complex structures well. The sys5 and sys6 are possibly struggling with long-range dependencies. All models fail to score by very long sentences (>=50 tokens). This might be caused by token truncation during preprocessing (max_length=250) or rare occurrence of long sentences in the dataset, and therefore insufficient for the training.

length	sys1	sys2	sys3	sys4	sys5	sys6
<10	5.05	4.91	4.91	9.37	9.41	3.03
[10,20)	15.49	14.10	14.10	15.87	14.64	15.24
[20,30)	9.07	9.07	9.07	8.24	8.28	7.96
[30,40)	7.85	5.68	5.68	8.26	8.17	8.18
[40,50)	30.65	**32.28**	**32.28**	24.90	20.95	22.41
[50,60)	0.00	0.00	0.00	0.00	0.00	0.00
>=60	0.00	0.00	0.00	0.00	0.00	0.00

English to French

The translation performance of the fine-tuned translation-based models was tested also on the models translating from English to French, which were fine-tuned on the checkpoint of a fine-tuned model translating from English to German. Similarly, each model was evaluated using the SacreBLEU metric on the test set, and further analysed using the *compare-mt* tool for deeper insights. Tables with all results are provided in the *Attachment 4*. Some of the results are explained below. The following references to the particular models were used:

- sys1: T5 model #3: „kammartina/t5_base_en-de-fr_finetuned_32K_cleaned_dataset“
- sys2: NLLB model #3: sanpoer/nllb_200_distilled_en-fr_finetuned_cleaned_32K

1. Aggregate BLEU scores

The overall BLEU scores:

	sys1	sys2
SacreBLEU score	12.74	**14.88**

v s1 / s2 ->	sys1	sys2
sys1		**-(p=0.06)**

The sys2 outperforms sys1 by +2.14 BLEU, and therefore indicates better overall translation quality. However, the p-value of 0.06 suggests that this difference is only marginally significant and does not meet the conventional 0.05 threshold for statistical significance.

2. **Sentence length and BLEU**

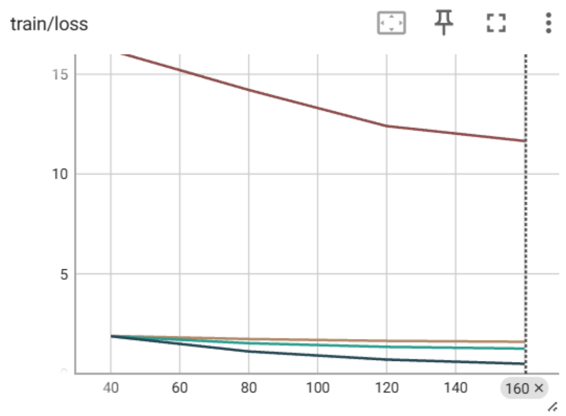
The following table provides again the BLEU scores given the sentence lengths. The scores highlight that sys2 outperforms sys1 in almost all buckets. The sys2 consistently achieves higher BLEU scores for sentences between 10 and 50 tokens. Both models perform poorly on very short (<10) and very long (>=60) sentences.

length	sys1	sys2
<10	5.65	5.26
[10,20)	7.39	**8.51**
[20,30)	18.66	**23.69**
[30,40)	5.07	**6.56**
[40,50)	21.79	**25.71**
[50,60)	0.00	0.00
>=60	1.61	0.71

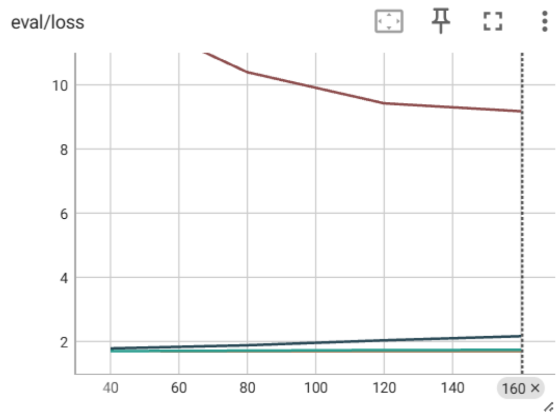
Model-agnostic Training Pipeline

To develop a training pipeline compatible with various transformer-based models, we implemented a script that reads from YAML configuration files specifying key parameters such as the model, source and target languages, learning rate, batch size, and more.

The pipeline is designed to be flexible, allowing users to configure options such as whether to save checkpoints. The training argument *save_strategy* can be set to *“epoch”* or *“no”*, depending whether early_stopping is applied. By combining *early_stopping* with *save_total_limits* we would retain only the most relevant checkpoints for loading the best-performing model to save disk space. The pipeline was tested with T5-base, NLLB-200, mT5, and mBART50 using a training set of 800 samples and a validation set of 100 samples. While mT5 (reddish-brown line in the charts) started with significantly higher training and evaluation losses, it demonstrated more noticeable improvement over time compared to the other models. This behavior may be attributed to the fact that mT5 has not been pre-trained on downstream tasks and may require substantially more data to perform competitively.



Run ↑	Value	Step	Relative
mBart50	0,4987	160	38.18 sec
mT5	11,6584	160	30.83 sec
nllb_200	1,2593	160	34.97 sec
t5	1,6012	160	24.06 sec

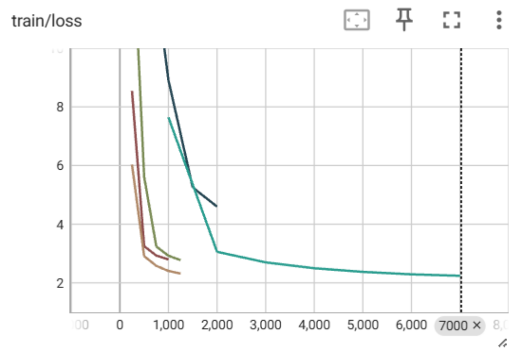


Run ↑	Value	Step	Relative
mBart50	2,17	160	41.37 sec
mT5	9,1775	160	37.68 sec
nllb_200	1,7447	160	58.3 sec
t5	1,6932	160	33.88 sec

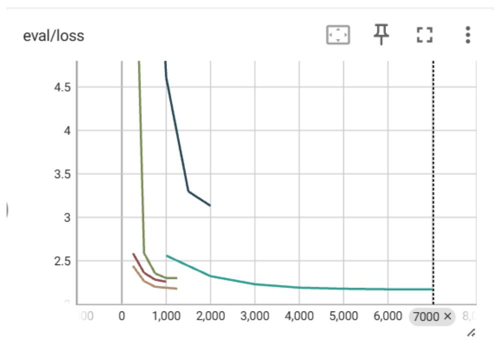
Based on the training pipeline, we created another script using Optuna for hyperparameter optimization and tested it on mT5 using 8,000 training samples and 1,000 validation samples. We incorporated *warmup_steps* for the learning rate, *max_length* for generation, and *num_beams* for beam search into the hyperparameter search space. An early stopping mechanism was also implemented to save the best model before the performance starts to decrease. As shown in the TensorBoard charts below, both training and validation losses decreased significantly. The best BLEU score of 16.69 is obtained from trial 3 and the following hyperparameters were identified as optimal:

learning_rate: 0.00014276083970070894,
weight_decay: 0.029286255489085245,
batch_size: 32,
num_epochs: 5,
warmup_steps: 46,
max_length: 135,
num_beams: 4

Development of train and validation losses of mT5 training across Optuna trials

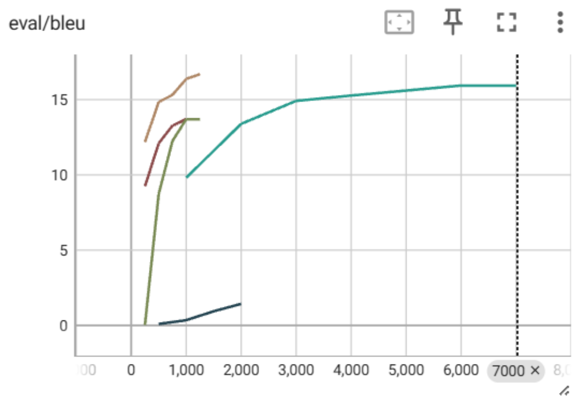


Run ↑	Value	Step	Relative
trial_0	4,6016	2 000	9.341 min
trial_1	2,7992	1 000	5.049 min
trial_2	2,2444	7 000	23.16 min
trial_3	2,3205	1 250	7.013 min
trial_4	2,7781	1 250	9.263 min



Run ↑	Value	Step	Relative
trial_0	3,1321	2 000	11.74 min
trial_1	2,2603	1 000	5.665 min
trial_2	2,1732	7 000	24.37 min
trial_3	2,1805	1 250	7.687 min
trial_4	2,3022	1 250	7.574 min

Development of validation BLEU scores of mT5 training across Optuna trials



	Run ↑	Value	Step	Relative
●	trial_0	1,4389	2 000	11.74 min
●	trial_1	13,7101	1 000	5.665 min
●	trial_2	15,9359	7 000	24.37 min
●	trial_3	16,6931	1 250	7.687 min
●	trial_4	13,6974	1 250	7.574 min

Conclusion

After fine-tuning on our training data, both T5-base and NLLB-200 showed substantial improvement over their pre-training performance in translating TED Talk sentences. BLEU scores on the test set indicate that bicleaning the training data and including the English–French direction improved translation quality for both models. When comparing them with each other, NLLB seems to perform slightly better than T5-base in all three variants. Nevertheless, analysis with compare-mt offered insights into word-level accuracy, sentence length matching, and translation performance across different sentence lengths. By combining these evaluation aspects, the baseline T5-base model trained on a pre-bicleaned dataset achieved the highest aggregate BLEU score, which also highlights the limitations of relying solely on a general BLEU score for evaluating machine translation quality. However, being English-centric and pretrained on limited languages, T5-base may not perform well on translations into English. For many-to-many translation, models trained on a diverse set of languages such as NLLB, which we fine-tuned in this project, may offer advantages in handling low-resource languages without relying on English as a pivot. Accordingly, we developed a training pipeline with extended hyperparameter tuning to support the exploration of other large multilingual models including mT5 and mBART50 and successfully conducted five trials with mT5 with a notable performance improvement.

We aimed to establish a statistical machine translation (SMT) baseline and completed tokenization and truecasing including training a truecasing model on a large corpus from the OPUS collection to standardize casing across the training data. Although GIZA++ and MGIZA compiled successfully, attempts to run them on the training data resulted in execution errors that could not be resolved within the project timeline. Additional complications arose due to missing packages that required administrator privileges (sudo) to install.

We observed that bicleaning the dataset contributed to improved training results for both models, even though this was not clearly reflected in the compare-mt evaluation. Therefore, we propose extracting Bicleaner's internal feature vectors to train a custom classifier for misalignment detection. This approach would allow fine-tuning to domain-specific noise patterns using models such as logistic regression or decision trees. By leveraging Bicleaner solely for feature extraction, without relying on its built-in filtering, we can incorporate labeled data or heuristic thresholds to better distinguish clean and noisy sentence pairs. This method offers greater flexibility and potential accuracy, especially in cases where off-the-shelf alignment models underperform due to domain mismatch or subtler types of noise.

References

Translation: https://huggingface.co/docs/transformers/tasks/translation

T5: https://huggingface.co/docs/transformers/model_doc/t5

SentencePiece: https://huggingface.co/docs/transformers/tokenizer_summary#sentencepiece

Metric: sacrebleu: https://huggingface.co/spaces/evaluate-metric/sacrebleu

google-t5/t5-base: https://huggingface.co/google-t5/t5-base

compare-mt: Because Scoring Your Systems Is Not Enough: https://medium.com/@bnjmn_marie/compare-mt-because-scoring-your-systems-is-not-enough-784d601872e6

compare-mt: https://github.com/neulab/compare-mt/tree/master

Attachments

Attachment 1

Model #1	Model #2	Model #3
MODEL NAME: **„t5_base_en-de_finetuned(32K)“** DIRECTORY (Hugging Face Hub): **https://huggingface.co/kammartina/t5_base_en-de_finetuned_32K**	MODEL NAME: **„t5_base_en-de_finetuned(32K)_cleaned_dataset“** DIRECTORY (Hugging Face Hub): **https://huggingface.co/kammartina/t5_base_en-de_finetuned_32K_cleaned_dataset**	MODEL NAME: **„t5_base_en-de-fr_finetuned(32K)_cleaned_dataset“** DIRECTORY (Hugging Face Hub): **https://huggingface.co/kammartina/t5_base_en-de-fr_finetuned_32K_cleaned_dataset**
NTB WITH CODE FOR FINETUNING: **„t5_base_en-de.ipynb“** DIRECTORY (GitHub): **“Finetuning/t5/t5_base_en_de_py”**	NTB WITH CODE FOR FINETUNING: **„t5_base_en-de.ipynb“** DIRECTORY (GitHub): **"Finetuning/t5/t5_base_en_de_py“**	NTB WITH CODE FOR FINETUNING: **„t5_base_en-fr.ipynb“** DIRECTORY (Github): **"Finetuning/t5/t5_base_en_fr_py“**

LANG. DIRECTION: **EN to DE**	LANG. DIRECTION: **EN to DE**	LANG. DIRECTION: **EN to DE, EN to FR**
MODEL LOADED: **„google-t5/t5-base“** DIRECTORY OF LOADED MODEL: **https://huggingface.co/google-t5/t5-base**	MODEL LOADED: **„google-t5/t5-base“** DIRECTORY OF LOADED MODEL: **https://huggingface.co/google-t5/t5-base**	MODEL LOADED: **" t5_base_en-de_finetuned(32K)_cleaned_dataset/checkpoint-8000"** DIRECTORY OF LOADED MODEL (Hugging Face Hub): **https://huggingface.co/kammartina/t5_base_en-de_finetuned_32K_cleaned_dataset**
TOKENIZER LOADED: **„google-t5/t5-base“** DIRECTORY OF LOADED TOKENIZER: **https://huggingface.co/google-t5/t5-base**	TOKENIZER LOADED: **„google-t5/t5-base“** DIRECTORY OF LOADED TOKENIZER: **https://huggingface.co/google-t5/t5-base**	TOKENIZER LOADED: **„google-t5/t5-base“** DIRECTORY OF LOADED TOKENIZER: **https://huggingface.co/google-t5/t5-base**
FILES USED: (format – json lines) **For finetuning, NOT cleaned files (with Bicleaner) were used.** **„train.jsonl“** **„val.jsonl“** **„test.jsonl“** FILES DIRECTORY (GitHub): **„Data/…“**	FILES USED: (format – json lines) **For finetuning, cleaned files (with Bicleaner) were used.** **„train.clean.jsonl“** **„val.clean.jsonl“** **„test.clean.jsonl“** FILES DIRECTORY (GitHub): **„Data/…“**	FILES USED: (format – json lines) **For finetuning, cleaned files (with Bicleaner) were used.** **„en-fr_train.clean.jsonl“** **„en-fr_val.clean.jsonl“** **„en-fr_test.clean.jsonl“** FILES DIRECTORY (GitHub): **„Data/…“**
TOTAL NUM OF SAMPLES (NOT cleaned dataset): `DatasetDict({ train: Dataset({ features: ['translation'], num_rows: 398421 }) })`	TOTAL NUM OF SAMPLES (cleaned dataset): `DatasetDict({ train: Dataset({ features: ['translation'], num_rows: 357746 }) })`	TOTAL NUM OF SAMPLES (cleaned dataset): `DatasetDict({ train: Dataset({ features: ['translation'], num_rows: 398392 }) })`

validation: Dataset({
features: ['translation'],
num_rows: 49803
})
test: Dataset({
features: ['translation'],
num_rows: 49803
})
})
	validation: Dataset({
	features: ['translation'],
	num_rows: 44724
	})
	test: Dataset({
	features: ['translation'],
	num_rows: 44789
	})
	})
		validation: Dataset({
		features: ['translation'],
		num_rows: 49760
		})
		test: Dataset({
		features: ['translation'],
		num_rows: 49758
		})
		})

SAMPLE OF TRAINING SET: Sample **of** the training **set**: {'translation': {'de': 'vom Atom, auf welches er trifft.', 'en': 'to the atom that it hits.'}} {'translation': {'de': '(Gelächter)', 'en': '(Laughter)'}} {'translation': {'de': 'Die Menschen unterstützen den Einsatz von Gewalt eher,', 'en': 'People are more likely to support the use'}}	SAMPLE OF TRAINING SET: Sample **of** the training **set**: {'translation': {'de': 'vom Atom, auf welches er trifft.', 'en': 'to the atom that it hits.'}} {'translation': {'de': 'Die Menschen unterstützen den Einsatz von Gewalt eher,', 'en': 'People are more likely to support the use'}} {'translation': {'de': 'und ihn wie einen Freund zu behandeln,', 'en': "That's why in negotiations, often, when things get " 'tough,'}}	SAMPLE OF TRAINING SET: Sample **of** the training set: {'translation': {'en': 'Every man deserves the opportunity', 'fr': 'Tous les hommes méritent l'opportunité'}} {'translation': {'en': 'and **do** it **in** terms **of** its affordability,', 'fr': 'et de faire cela du point de vue de son prix'}} {'translation': {'en': 'Video: (Police siren **and** shouting over megaphone)', 'fr': "Nous voyons qu'il y a des palestiniens qui souffrent "'beaucoup, qui ont perdu leurs enfants,'}}

NUM OF SAMPLES FOR FINETUNING:
`After subsampling:`
`{`
`'test': 512,`
`'train': 32000,`
`'validation': 512`
`}`
	NUM OF SAMPLES FOR FINETUNING:
	`After subsampling:`
	`{`
	`'test': 512,`
	`'train': 32000,`
	`'validation': 512`
	`}`
		NUM OF SAMPLES FOR FINETUNING:
		`After subsampling:`
		`{`
		`'test': 512,`
		`'train': 32000,`
		`'validation': 512`
		`}`

PREPROCESSING FUNCTION:
`source_lang = "en"`
`target_lang = "de"`
`prefix = "translate English to German: "`
`max_length=250`
	PREPROCESSING FUNCTION:
	`source_lang = "en"`
	`target_lang = "de"`
	`prefix = "translate English to German: "`
	`max_length=250`
		PREPROCESSING FUNCTION:
		`source_lang = "en"`
		`target_lang = "fr"`
		`prefix = "translate English to French: "`
		`max_length=250`

EVALUATION METRICS: **SacreBLEU** SOURCE CODE FOR THIS METRIC: https://huggingface.co/docs/transformers/tasks/translation#evaluate	EVALUATION METRICS: **SacreBLEU** SOURCE CODE FOR THIS METRIC: https://huggingface.co/docs/transformers/tasks/translation#evaluate	EVALUATION METRICS: **SacreBLEU** SOURCE CODE FOR THIS METRIC: https://huggingface.co/docs/transformers/tasks/translation#evaluate

HYPERPARAMETERS:	HYPERPARAMETERS:	HYPERPARAMETERS:

training_args = Seq2SeqTrainingArguments(
output_dir=os.path.join("/content/drive/MyDrive/ML2_Bublin_Project/Martina_T5/t5_base_en-fr_finetuned(32K)_cleaned_dataset"),
num_train_epochs=4,
per_device_train_batch_size=16,
per_device_eval_batch_size=16,
eval_strategy="epoch",
save_strategy="epoch",
#logging_steps=100,
learning_rate=2e-5,
weight_decay=0.01,
fp16=True,
report_to="none",
load_best_model_at_end=True,
predict_with_generate=True,
generation_max_length=250,
generation_num_beams=5,
)
	training_args = Seq2SeqTrainingArguments(
	output_dir=os.path.join("/content/drive/MyDrive/ML2_Bublin_Project/Martina_T5/t5_base_en-fr_finetuned(32K)_cleaned_dataset"),
	num_train_epochs=4,
	per_device_train_batch_size=16,
	per_device_eval_batch_size=16,
	eval_strategy="epoch",
	save_strategy="epoch",
	#logging_steps=100,
	learning_rate=2e-5,
	weight_decay=0.01,
	fp16=True,
	report_to="none",
	load_best_model_at_end=True,
	predict_with_generate=True,
	generation_max_length=250,
	generation_num_beams=5,
	)
		training_args = Seq2SeqTrainingArguments(
		output_dir=os.path.join("/content/drive/MyDrive/ML2_Bublin_Project/Martina_T5/t5_base_en-fr_finetuned(32K)_cleaned_dataset"),
		num_train_epochs=4,
		per_device_train_batch_size=16,
		per_device_eval_batch_size=16,
		eval_strategy="epoch",
		save_strategy="epoch",
		#logging_steps=100,
		learning_rate=2e-5,
		weight_decay=0.01,
		fp16=True,
		report_to="none",
		load_best_model_at_end=True,
		predict_with_generate=True,
		generation_max_length=250,
		generation_num_beams=5,
		)

<table>
<tr>
<td>TRAINING LOGS:

[8000/8000 31:58, Epoch 4/4]
<table>
<tr><th>Epoch</th><th>Training Loss</th><th>Validation Loss</th><th>Bleu</th><th>Gen Len</th></tr>
<tr><td>1</td><td>1.846500</td><td>1.710968</td><td>21.133400</td><td>24.687500</td></tr>
<tr><td>2</td><td>1.780200</td><td>1.697086</td><td>21.338600</td><td>24.406200</td></tr>
<tr><td>3</td><td>1.738900</td><td>1.693702</td><td>21.300200</td><td>24.375000</td></tr>
<tr><td>4</td><td>1.734500</td><td>1.691780</td><td>21.306000</td><td>24.437500</td></tr>
</table>
TrainOutput(

global_step=8000,

training_loss=1.7891621322631837,

metrics={

'train_runtime': 1918.3047,

'train_samples_per_second': 66.726,

'train_steps_per_second': 4.17,

'total_flos': 3713653575843840.0,

'train_loss': 1.7891621322631837,

'epoch': 4.0}

)</td>
<td>TRAINING LOGS:

[8000/8000 32:06, Epoch 4/4]
<table>
<tr><th>Epoch</th><th>Training Loss</th><th>Validation Loss</th><th>Bleu</th><th>Gen Len</th></tr>
<tr><td>1</td><td>1.775700</td><td>1.683444</td><td>22.186000</td><td>24.812500</td></tr>
<tr><td>2</td><td>1.714700</td><td>1.671195</td><td>22.480700</td><td>24.656200</td></tr>
<tr><td>3</td><td>1.644200</td><td>1.671071</td><td>22.274200</td><td>24.531200</td></tr>
<tr><td>4</td><td>1.637900</td><td>1.669474</td><td>22.357600</td><td>24.687500</td></tr>
</table>
TrainOutput(

global_step=8000,
training_loss=1.7076805419921874,

metrics={

'train_runtime': 1926.7165,

'train_samples_per_second': 66.434,

'train_steps_per_second': 4.152,

'total_flos': 3828936916500480.0,

'train_loss': 1.7076805419921874,

'epoch': 4.0}

)</td>
<td>TRAINING LOGS:

[8000/8000 31:27, Epoch 4/4]
<table>
<tr><th>Epoch</th><th>Training Loss</th><th>Validation Loss</th><th>Bleu</th><th>Gen Len</th></tr>
<tr><td>1</td><td>1.335800</td><td>1.207630</td><td>32.295900</td><td>30.406200</td></tr>
<tr><td>2</td><td>1.310900</td><td>1.200040</td><td>32.240600</td><td>30.656200</td></tr>
<tr><td>3</td><td>1.254200</td><td>1.195104</td><td>32.394500</td><td>30.406200</td></tr>
<tr><td>4</td><td>1.241000</td><td>1.195033</td><td>32.691800</td><td>30.468800</td></tr>
</table>
TrainOutput(

global_step=8000,

training_loss=1.290485107421875,

metrics={

'train_runtime': 1888.5828,

'train_samples_per_second': 67.776,

'train_steps_per_second': 4.236,

'total_flos': 3803398744965120.0,

'train_loss': 1.290485107421875,

'epoch': 4.0}

)</td>
</tr>
<tr>
<td>PREDICTION ON VAL SET:

Forgot to track this result

DIRECTORY txt-FILE (GitHub):

„Finetuning/t5/t5_val_test_set_results.txt“</td>
<td>PREDICTION ON VAL SET:

Validation results: {

'test_loss': 1.6694741249084473,

'test_bleu': 22.3576,

'test_gen_len': 24.6875,

'test_runtime': 26.7633,

'test _samples_per_second': 19.131,

'test_steps_per_second': 1.196

}

DIRECTORY txt-FILE (GitHub):</td>
<td>PREDICTION ON VAL SET:

Validation results: {

'test_loss': 1.1950327157974243,

'test_bleu': 32.6918,

'test_gen_len': 30.4688,

'test_runtime': 29.7915,

'test_samples per_second': 17.186,

'test_steps per_second': 1.074

}

DIRECTORY txt-FILE (GitHub):</td>
</tr>
</table>

	„**Finetuning/t5/t5_val_test_set_results.txt**“	„**Finetuning/t5/t5_val_test_set_results.txt**“
EVALUATION ON TEST SET (epoch 3; checkpoint-6000): **Test results: {** **'eval_loss': 1.7099956274032593,** **'eval_model_preparation_time': 0.0051,** **'eval_bleu': 22.0863,** **'eval_gen_len': 25.625,** **'eval_runtime': 26.3846,** **'eval_samples_per_second': 19.405,** **'eval_steps_per_second': 1.213** **}** DIRECTORY txt-FILE (GitHub): „**Finetuning/t5/t5_val_test_set_results.txt**“	EVALUATION ON TEST SET (epoch 4; checkpoint-8000): **Test results: {** **'eval_loss': 1.579579472541809,** **'eval_model_preparation_time': 0.0052,** **'eval_bleu': 24.2424,** **'eval_gen_len': 25.875,** **'eval_runtime': 28.4713,** **'eval_samples_per_second': 17.983,** **'eval_steps_per_second': 1.124** **}** DIRECTORY txt-FILE (GitHub): „**Finetuning/t5/t5_val_test_set_results.txt**“	EVALUATION ON TEST SET (epoch 4; checkpoint-8000): **Test results: {** **'eval_loss': 1.3253144025802612,** **'eval_model_preparation_time': 0.0052,** **'eval_bleu': 27.9185,** **'eval_gen_len': 34.0938,** **'eval_runtime': 35.9594,** **'eval_samples_per_second': 14.238,** **'eval_steps_per_second': 0.89** **}** DIRECTORY txt-FILE (GitHub): „**Finetuning/t5/t5_val_test_set_results.txt**“
COMPARISON WITH OTHER MODELS USING „*compare-mt*“ TOOL	COMPARISON WITH OTHER MODELS USING „*compare-mt*“ TOOL	COMPARISON WITH OTHER MODELS USING „*compare-mt*“ TOOL

Attachment 2

Model(s): NLLB

Model #1	Model #2	Model #3
MODEL NAME: **„nllb_en-de_finetuned(32K)“** DIRECTORY (Hugging Face): https://huggingface.co/sanpoer/nllb_200_distilled_en-de_finetuned	MODEL NAME: **„nllb_en-de_finetuned(32K)_cleaned_dataset“** DIRECTORY (Hugging Face): https://huggingface.co/sanpoer/nllb_200_distilled_en_de_finetuned_cleaned32K/tree/main	MODEL NAME: **„nllb_en-fr_finetuned(32K)_cleaned_dataset“** DIRECTORY (Hugging Face): https://huggingface.co/sanpoer/nllb_200_distilled_en-fr_finetuned_cleaned_32K
NTB WITH CODE FOR FINETUNING: **„nllb_en-de.ipynb“** DIRECTORY (GitHub): **“Finetuning/nllb/nllb_en_de.py”**	NTB WITH CODE FOR FINETUNING: **„nllb_en-de.ipynb“** DIRECTORY (GitHub): **“Finetuning/nllb/nllb_en_de.py”**	NTB WITH CODE FOR FINETUNING: **„nllb_en-fr.ipynb“** DIRECTORY (GitHub): **“Finetuning/nllb/nllb_en_fr.py”**
LANG. DIRECTION: **EN to DE**	LANG. DIRECTION: **EN to DE**	LANG. DIRECTION: **EN to DE, EN to FR**
MODEL LOADED: **„facebook/nllb-200-distilled-600M“** DIRECTORY OF LOADED MODEL: https://huggingface.co/facebook/nllb-200-distilled-600M	MODEL LOADED: **„facebook/nllb-200-distilled-600M“** DIRECTORY OF LOADED MODEL: https://huggingface.co/facebook/nllb-200-distilled-600M	MODEL LOADED: **"nllb_en-de_finetuned(32K)_cleaned_datasetcheckpoint-10000"** DIRECTORY OF LOADED MODEL (Hugging Face): https://huggingface.co/sanpoer/nllb_200_distilled_en_de_finetuned_cleaned32K
TOKENIZER LOADED: **„facebook/nllb-200-distilled-600M“** DIRECTORY OF LOADED TOKENIZER:	TOKENIZER LOADED: **„facebook/nllb-200-distilled-600M“** DIRECTORY OF LOADED TOKENIZER:	TOKENIZER LOADED: **"nllb_en-de_finetuned(32K)_cleaned_datasetcheckpoint-10000"** DIRECTORY OF LOADED TOKENIZER (Hugging Face): https://huggingface.co/sanpo

https://huggingface.co/facebook/nllb-200-distilled-600M	https://huggingface.co/facebook/nllb-200-distilled-600M	er/nllb_200_distilled_en_de_finetuned_cleaned32K
FILES USED: (format – json lines) **For finetuning, NOT cleaned files (with Bicleaner) were used.** **„train.jsonl“** **„val.jsonl“** **„test.jsonl“** FILES DIRECTORY (GitHub): **„Data/…“**	FILES USED: (format – json lines) **For finetuning, cleaned files (with Bicleaner) were used.** **„train.clean.jsonl“** **„val.clean.jsonl“** **„test.clean.jsonl“** FILES DIRECTORY (GitHub): **„Data/…“**	FILES USED: (format – json lines) **For finetuning, cleaned files (with Bicleaner) were used.** **„en-fr_train.clean.jsonl“** **„en-fr_val.clean.jsonl“** **„en-fr_test.clean.jsonl“** FILES DIRECTORY (GitHub): **„Data/…“**

TRAINING LOGS:

```
Starting training...
```

[1000

Epoch	Training Loss	Validation Loss	Sa
1	0.574700	0.720737	2
2	0.550800	0.707716	2
3	0.543700	0.705601	2
4	0.531200	0.705226	2
5	0.521900	0.705388	2

TrainOutput(

global_step=10000,

training_loss=0.9618924423217774,

metrics={

'train_runtime': 5225.7376,

'train_samples_per_second': 30.618,

'train_steps_per_second': 1.914,

'total_flos': 1.18513532928e+16,

'train_loss': 0.9618924423217774,

'epoch': 5.0}

)

TRAINING LOGS:

```
Starting training...
```

[10000

Epoch	Training Loss	Validation Loss	Sa
1	0.648600	0.736672	2
2	0.610100	0.723648	2
3	0.565800	0.721524	2
4	0.571300	0.720791	2
5	0.567400	0.720630	2

TrainOutput (

global_step=10000,

training_loss=0.9887595901489258,

metrics={

'train_runtime': 4978.3608,

'train_samples_per_second': 32.139,

'train_steps_per_second': 2.009,

'total_flos': 1.286718357504e+16,

'train_loss': 0.9887595901489258,

'epoch': 5.0}

)

TRAINING LOGS:

```
Starting training...
```

[100

Epoch	Training Loss	Validation Loss	S
1	0.439900	0.505471	
2	0.426400	0.502898	
3	0.393600	0.502064	
4	0.398200	0.502229	
5	0.385600	0.502655	

TrainOutput (

global_step=10000,

training_loss=0.41453202590942384,

metrics={

'train_runtime': 5243.0737,

'train _samples_per_second' : 30.516,

'train_steps_per_second': 1.907,

'total _flos': 1.18513532928e+16,

'train_loss': 0.41453202590942384,

'epoch': 5.07}

)

PREDICTION ON VAL SET: **Validation results: {** **'test_loss': 0.705388069152832,** **'test_sacrebleu': 21.366065417680694,** **'test_chrf': 45.79612563428759,** **'test_runtime': 24.1698,** **'test_samples_per_second': 21.183,** **'test_steps_per_second': 1.324** **}** DIRECTORY txt-FILE (server): **„Finetuning/nllb/nllb_val_test_set_results.txt“**	PREDICTION ON VAL SET: **Validation results: {** **'test_loss': 0.7206302285194397,** **'test_sacrebleu': 22.799425272001375,** **'test_chrf': 46.836292856784056,** **'test_runtime': 22.39,** **'test_samples_per_second': 22.867,** **'test_steps_per_second': 1.429** **}** DIRECTORY txt-FILE (server): **„Finetuning/nllb/nllb_val_test_set_results.txt“**	PREDICTION ON VAL SET: **Validation results: {** **'test_loss': 0.9140143394470215,** **'test_model_preparation_time': 0.0056,** **'test_sacrebleu': 1.0275933096614676,** **'test_chrf': 15.066150447584217,** **'test_runtime': 28.4553,** **'test_samples_per_second': 17.993,** **'test_steps_per_second': 1.125** **}** DIRECTORY txt-FILE (server): **„Finetuning/nllb/nllb_val_test_set_results.txt“**

EVALUATION ON TEST SET (epoch 4; checkpoint-8000): **Test results: {** **'eval_loss': 0.7935734391212463,** **'eval_model_preparation_time': 0.006,** **'eval_sacrebleu': 23.25238160521439,** **'eval_chrf': 46.57877856170928,** **'eval_runtime': 22.9581,** **'eval_samples_per_second': 22.302,** **'eval_steps_per_second': 1.394** **}** SCREENSHOT DIRECTORY (Drive): **„/content/drive/MyDrive/ML2_Bublin_Project/Sandra_NLLB/nllb_en-de_finetuned(32K)/Evaluation on the test set (nllb_en-de_finetuned(32K)) - epoch 4, checkpoint 8000“** DIRECTORY txt-FILE (server): **„Finetuning/nllb/nllb_val_test_set_results.txt“**	EVALUATION ON TEST SET (epoch 5; checkpoint-10000): **Test results: {** **'eval_loss': 0.7263513207435608,** **'eval_model_preparation_time': 0.0058,** **'eval_sacrebleu': 25.199590318925576,** **'eval_chrf': 49.433325214598106,** **'eval_runtime': 22.9602,** **'eval_samples_per_second': 22.299,** **'eval_steps_per_second': 1.394** **}** SCREENSHOT DIRECTORY (Drive): **"/content/drive/MyDrive/ML2_Bublin_Project/Sandra_NLLB/nllb_en-de_finetuned(32K)_cleaned_dataset/Evaluation on the test set (nllb_en-de_finetuned(32K)_cleaned_dataset) - epoch 5, checkpoint 10000“** DIRECTORY txt-FILE (server): **„Finetuning/nllb/nllb_val_test_set_results.txt“**	EVALUATION ON TEST SET (epoch 5; checkpoint-10000): **Test results: {** **'eval_loss': 0.579248309135437,** **'eval_model_preparation_time': 0.0056,** **'eval_sacrebleu': 32.33813227420938,** **'eval_chrf': 53.44186240157883,** **'eval_runtime': 22.9247,** **'eval_samples_per_second': 22.334,** **'eval_steps_per_second': 1.396** **}** SCREENSHOT DIRECTORY (Drive): **"/content/drive/MyDrive/ML2_Bublin_Project/Sandra_NLLB/nllb_en-fr_finetuned(32K)_cleaned_dataset/Evaluation on the test set (nllb_en-fr_finetuned(32K)_cleaned_dataset) - epoch 5, checkpoint 10000“** DIRECTORY txt-FILE (server): **„Finetuning/nllb/nllb_val_test_set_results.txt“**
COMPARISON WITH OTHER MODELS USING *„compare-mt“* TOOL	COMPARISON WITH OTHER MODELS USING *„compare-mt“* TOOL	COMPARISON WITH OTHER MODELS USING *„compare-mt“* TOOL

Attachment 3

Word accuracy analysis – word frequency measures by frequency buckets:

frequency	sys1	sys2	sys3	sys4	sys5	sys6
<1	0.00	0.00	0.00	0.00	0.00	0.00
1	0.32	0.31	0.31	0.31	0.28	0.30
2	0.44	0.42	0.42	0.39	0.40	0.41
3	0.44	0.46	0.46	0.43	0.46	0.48
4	0.36	0.40	0.40	0.53	0.49	0.44
[5,10)	0.42	0.41	0.41	0.41	0.40	0.40
[10,100)	0.54	0.53	0.53	0.51	0.51	0.52
[100,1000)	0.00	0.00	0.00	0.00	0.00	0.00
>=1000	0.00	0.00	0.00	0.00	0.00	0.00

The compare-mt frequency bucket analysis showed how well each system translated words of varying frequency. Accuracy for rare words (frequency 1–4) was modest across all models, peaking in mid-frequency ranges. The EN→FR model (sys6) matched the EN→DE systems in common word buckets ([10–100]) but struggled slightly with very rare or frequent terms. All models scored 0.00 for extremely frequent terms (≥1000), likely due to preprocessing or tokenizer filtering.

Differences in number of tokens between the reference and machine translation outputs lengths (len(output)-len(reference)):

lengthdiff	sys1	sys2	sys3	sys4	sys5	sys6
<-20	1	1	1	1	1	1
[-20,-10)	3	3	3	3	3	3
[-10,-5)	6	6	6	5	5	5
-5	3	3	3	3	4	3
-4	1	1	1	2	1	1
-3	1	1	1	1	1	2
-2	3	1	1	2	1	2

-1	2	2	2	4	3	0
0	7	9	9	5	7	9
1	4	5	5	7	7	5
2	4	3	3	6	8	8
3	5	4	4	6	1	2
4	6	4	4	4	6	6
5	3	6	6	1	3	4
[6,11)	10	10	10	5	5	5
[11,21)	1	2	2	6	5	5
>=21	3	2	2	2	2	2

This analysis measures fluency and alignment. Most models had a strong clustering around 0 difference, indicating well-calibrated output lengths. Few outliers existed in the extreme buckets (e.g., output longer by more than 20 tokens), showing stability in length prediction across systems.

Sentence-level BLEU:

	sys1	sys2	sys3	sys4	sys5	sys6
<10.0	29	30	30	28	28	31
[10.0,20.0)	12	10	10	13	13	11
[20.0,30.0)	8	10	10	6	11	8
[30.0,40.0)	5	5	5	9	4	5
[40.0,50.0)	2	1	1	4	4	2
[50.0,60.0)	2	2	2	0	0	2
[60.0,70.0)	1	1	1	0	0	0
[70.0,80.0)	4	4	4	3	3	4
[80.0,90.0)	0	0	0	0	0	0
>=90.0	0	0	0	0	0	0

BLEU scores varied across sentence lengths. Shorter sentences (<10 tokens) had the lowest BLEU scores across all models. The highest BLEU values were observed for medium-length sentences (40–50 tokens), especially in the cleaned EN→DE model (sys2: 32.28 BLEU). Very long sentences (>60 tokens) were rare and typically underperformed or absent.

Attachment 4

Word accuracy analysis – word frequency measures by frequency buckets

frequency	sys1	sys2
<1	0.00	0.00
1	0.19	0.20
2	0.28	0.30
3	0.36	0.32
4	0.26	0.24
[5,10)	0.31	0.33
[10,100)	0.31	0.35
[100,1000)	0.00	0.00
>=1000	0.00	0.00

The sys1 performs consistently better or equal across all frequency ranges, particularly in the [10–100] token bucket, which typically includes general-purpose vocabulary. This suggests that the FR-specific training led to more precise vocabulary usage.

Differences in number of tokens between the reference and machine translation outputs lengths (len(output)-len(reference)):

lengthdiff	sys1	sys2
<-20	5	5
[-20,-10)	9	8
[-10,-5)	1	1
-5	1	1
-4	1	1
-3	1	1
-2	1	0
-1	4	3
0	5	4
1	6	4

2	1	4
3	1	4
4	4	3
5	3	3
[6,11)	6	7
[11,21)	6	8
>=21	5	3

Both models have similar error profiles regarding length mismatches. The sys1 shows slightly fewer extreme under-generation cases (e.g., fewer in <-20 bucket). The sys1 also has more balanced output, with slightly more matches at 0–4 token difference. The dedicated FR fine-tuning in sys1 may help the model better match expected sentence length, improving fluency and alignment.

Sentence-level BLEU:

	sys1	sys2
<10.0	39	36
[10.0,20.0)	5	8
[20.0,30.0)	6	7
[30.0,40.0)	3	2
[40.0,50.0)	4	4
[50.0,60.0)	1	1
[60.0,70.0)	0	0
[70.0,80.0)	2	1
[80.0,90.0)	0	1
>=90.0	0	0

The sys1 has fewer very low BLEU translations and more medium-to-high BLEU ones, indicating a broader spread of well-translated sentences, whereas sys2 clusters more heavily in the lowest BLEU range.